

Fermat's Little Theorem and Cryptography

John Gupta-She



1 Introduction

Computer science is math. Everything about computer science is math. Computer science is just another form of math, one that lets us handle bigger numbers that we'd otherwise be scared of. Of course, encryption is no different than the rest of computer science: it uses a certain concept in math, number theory. More specifically, the concept of encryption is the concept of Fermat's Little Theorem (FLT).

2 Fermat's Little Theorem

Of course, to understand how FLT relates to encryption, we have to explore it in detail first. FLT is a basic, but essential, number theory concept that uses prime numbers' unique condition of being relatively prime to all other natural numbers. The statement goes like this:

If p is a prime, then

$$a^{p-1} \equiv 1 \pmod{p}$$

This is a nontrivial fact, so it must be proved. The proof is quite clever, and as mentioned before, leverages the fact that primes are relatively prime to all other natural numbers.

2.1 Proof

Consider the sets $\{1, 2, 3, \dots, p-1\}$ and $\{a, 2a, 3a, \dots, (p-1)a\}$. I claim that these sets are equivalent mod p as long as a is relatively prime to p . To prove that, we can consider the second set. We know the first set is all the numbers from 1 through $p-1$, so we just need to prove that the second set is equal to that, mod p . With mod p , we already know that if there are no repeats, then there are only p different possibilities for elements of this set: 0 through $p-1$. We also cannot get 0, because that implies one of ja is divisible by p , where j is a random number from 1 through $p-1$. This means that if all the elements of $\{a, 2a, 3a, \dots, (p-1)a\}$ are different, we will have $p-1$ distinct elements to fill $p-1$ spaces (namely 1 through $p-1$), which is exactly what we want. Now let us prove that all terms of $\{a, 2a, 3a, \dots, (p-1)a\}$ are distinct. Let us assume that two elements are congruent mod p . Let us call them ia and ja . That means that

$$ia \equiv ja \pmod{p}$$

$$ia - ja \equiv 0 \pmod{p}$$

Since a is relatively prime to p , we can divide by a on both sides:

$$i - j \equiv 0 \pmod{p}$$

Since i and j are each between 1 and $p-1$, we know that their difference is between $-(p-2)$ and $p-2$. The only number in this range that is $0 \pmod{p}$ is 0, so their difference must be 0. This means that $i - j = 0$, so $i = j$. This is a contradiction, since we assumed that these were two different elements of $\{a, 2a, 3a, \dots, (p-1)a\}$, but these are the same element. Thus, our original assumption that there are two elements that are the same mod p is incorrect, so they must all be different mod p . Remember that this proves that the set $\{a, 2a, 3a, \dots, (p-1)a\}$ is really the same as the set $\{1, 2, 3, \dots, p-1\}$. We know that two equivalent sets mod p must have the same product mod p . This gives

$$(p-1)!a^{p-1} \equiv (p-1)! \pmod{p}$$

Since all of the numbers from 1 to $p-1$ are relatively prime to p , we can divide by $(p-1)!$ on both sides, resulting in:

$$a^{p-1} \equiv 1 \pmod{p}$$

Thus, we have arrived at the theorem statement, so the proof is completed.

2.2 Euler's Generalization

There is a generalized version of FLT that does not rely on the mod being prime, but rather that the base (a) and the mod (let's call it k) must be relatively prime. It is a proof that is rather similar to the proof I presented here, so I will leave it as an exercise. Just remember the main points of the previous proof: two equivalent sets mod k that are also a multiple of a away from each other. In the end, instead of the power being a simple expression like $p-1$, it becomes the output of a function on k , that we call Euler's totient function, denoted as $\phi(k)$.

3 Connections to Encryption

It's difficult to make a clear connection to encryption until you really apply this concept. Let's explore an encryption scenario that is strong because of its usage of FLT.

Say we want to send a message that is an integer mod N . Choose an integer a from between 1 and N that is relatively prime to $\phi(N)$. The public key is made up of N and a where a is called an encryption key.

At this point, anyone can use this public key to encrypt messages to the creator of the public key, because the creator will have a private key to decode messages. To encode the message, take a number you wanted to encrypt, let's call it M , raise it to the a th power, and take it mod N . This would make a new number mod N that seemingly has no relation to M , whatsoever. The new number would be

$$A(M) \equiv M^a \pmod{N}$$

where A is some sort of function.

To decode this, we need to determine what this private key will look like. The private key, b , should satisfy the condition

$$a \cdot b \equiv 1 \pmod{\phi(N)}$$

This implies that

$$a \cdot b = \phi(N) \cdot k + 1$$

Now to decode $F(M)$ back into M , mod N , we raise $F(M)$ to the b power, and take it mod N again. Let the function $B(x)$ equal taking X to the b power mod N . We get

$$B(A(M)) \equiv (M^a)^b \pmod{N}$$

$$(M^a)^b \equiv M^{a \cdot b} \pmod{N}$$

From our previous equation we can replace $a \cdot b$ with $\phi(N) \cdot k + 1$.

$$M^{a \cdot b} \equiv M^{\phi(N) \cdot k + 1} \pmod{N}$$

$$M^{\phi(N) \cdot k + 1} \equiv (M^{\phi(N)})^k \cdot M \pmod{N}$$

We know that $M^{\phi(N)}$ is equivalent to 1 mod N , so

$$(M^{\phi(N)})^k \cdot M \equiv (1)^k \cdot M \equiv M \pmod{N}$$

This shows that $B(A(M)) \equiv M \pmod{N}$, so we do have a valid private key. This all works because of Euler's totient function and the fact that there is an inverse to a . However, finding an inverse to a is not quite as simple if you don't already know the prime factorization of N , like the creator of the public key does.

3.1 Difficulty in Factoring

For someone other than the creator of the public key, they would have to invert $a, \text{mod}(\phi(N))$ to find the private key. Remember, $\phi(N)$ is dependent on the prime factorization of N . For a very large N , this prime factorization can be nearly impossible to find. Think about the amount of calculation needed to prime factor a number. First, you need to check if it is divisible by every prime up until the number's square root. This takes days upon days on its own for a large N . What makes it even worse is that you have to do this process over and over until you know you have it completely prime factored. This takes an absurdly long amount of time and is therefore not a viable way to crack the code.

Without this prime factorization of N , we cannot find $\phi(N)$ or find the inverse of $a(\text{mod}\phi(N))$. In the end, we cannot decrypt the code this way.

3.2 Some of the Largest Primes

Below are some of the largest primes we know currently, so you can see that it will take days and days, even for the world's strongest supercomputer. From largest to smallest, these are the five largest primes we know of:

1. $2^{82589933} - 1$
2. $2^{77232917} - 1$
3. $2^{74207281} - 1$
4. $2^{57885161} - 1$
5. $2^{43112609} - 1$

An interesting pattern we have is that they are all one less than some power of 2. This actually is not a coincidence. These kinds of primes ($2^n - 1$) are called Mersenne primes. The reason we have so many inconceivably large Mersenne primes is because there is actually an organization dedicated to finding the largest Mersenne primes. The organization's name is GIMPS, which stands for Great Internet Mersenne Prime Search. Anyone can download their free software and start looking for Mersenne primes on their own. The part I find the most funny is that the URL for their website is simply www.mersenne.org. Anyway, this organization is responsible for finding much more than just the largest five primes. They are responsible for the largest eight primes, but responsible for the last fifteen record holders for largest prime at their times.

The numbers themselves aren't too interesting, but to really grasp how large these numbers are, you have to look at the number of digits they have. Here we have the number of digits of the first 10 largest primes:

1. 24,862,048
2. 23,249,425
3. 22,338,618
4. 17,425,170
5. 12,978,189
6. 12,837,064

7. 11,185,272
8. 9,808,358
9. 9,383,761
10. 9,152,052

Going back to the original point of this tangent about large primes, even if we choose only two of these extremely large primes, we are already in the millions of digits. That means to decrypt the message, a person would have to check every single prime up until the square root of N , which should also have millions of digits.

4 Final Thoughts

All in all, FLT is an important outcome of number theory, not only for further advances in mathematics, but also further advances in cryptography. The use of this theorem makes it possible to encrypt messages with the use of primes, by using the fact that factoring is not as simple as multiplying (as far as we know). When primes become monstrously large, prime factoring larger numbers is not an easy task, which is the reason this form of encryption is so powerful.

And of course, what kind of article would this be without leaving you with some questions to think about? And here they are: What are some other ways you could leverage these concepts to encrypt information? Is it possible to use a different procedure while still using the core idea of not being able to easily factor large numbers? What would such a procedure look like?

I encourage you all to look more into this field of cryptography. Understanding how your privacy is protected is important, so you know your critical information is actually safe.

5 Problems

1. What are the last two digits of $7^{39} + 9^{41} + 11^{43}$?
2. Define $f(x)$ as x^{x^x} . What is $f(18) + f(19) + f(20) \pmod{91}$?
3. Find $111\dots 111$ (97 ones) $\pmod{97}$.
4. Find the smallest positive integer n such that n^{93} and 93^n leave the same remainder when divided by 100. (Art of Problem Solving)
5. Find all positive integers n between 0 and 1000 such that $n^{100} \equiv 1 \pmod{19}$.
6. One of Euler's conjectures was disproved in the 1960s by three American mathematicians when they showed there was a positive integer such that $133^5 + 110^5 + 84^5 + 27^5 = n^5$. Find the value of n . (AIME 1989 #9)

References

“Great Internet Mersenne Prime Search - PrimeNet.” 2^p-1 , August 4, 2020. <https://www.mersenne.org/>.

“Largest Known Prime Number.” Wikipedia. Wikimedia Foundation, January 13, 2021.
https://en.wikipedia.org/wiki/Largest_known_prime_number